



模块二 · 企业级 Agentic AI 落地与 Harness 架构实战

深入解构 Agent Harness 运行时与治理中枢

50 分钟 · 从能跑通的 demo 到跑得住的生产

配套仓库：github.com/kinfey/EnterpriseAgenticWorkshop

上一模块的结论

ReAct 能跑通 demo, 却跑不进企业生产。

能自主思考只是起点；一旦这套循环去调用真实业务系统，非确定性、无状态、越权与死循环会立刻把它挡在生产门外。

本模块用三条主线讲透 Harness 治理中枢

01 架构解耦

先分清 Model 与 Harness 各自的职责边界

02 五大支柱

上下文 · 工具 · 状态 · 安全 · 人机协同

03 仓库映射

落到 Brain / Hands / Session / Vault 四层

三条主线层层递进：先讲清为什么要解耦，再拆开五大治理支柱，最后回到真实仓库看它如何落地。

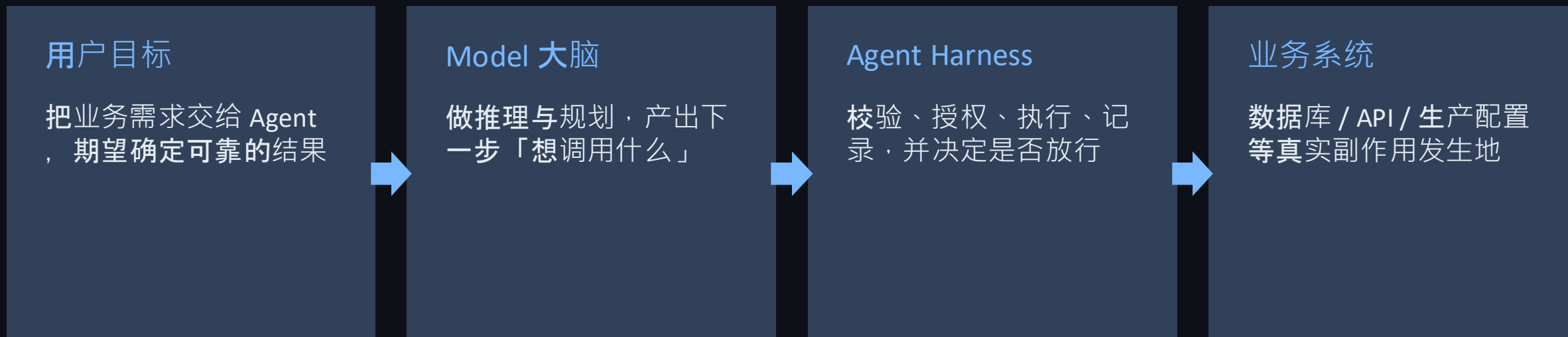


第一段 · 架构解耦

架构解耦：Model 是大脑，
Harness 是操作系统

把「会思考」与「能安全运行」彻底分开

Model 只负责思考，把 Agent 真正托住的是 Harness



Model 只输出「意图」，任何落到业务系统的「动作」都必须穿过 Harness 这一层。

来源：仓库 AgentHarness_HostedAgent 架构说明

大脑给意图，骨架给约束，两者职责必须彻底分离

Model 的职责 · 大脑

理解任务、多步规划、选择工具与参数、根据观测调整策略。它擅长的是「在不确定中给出下一步」，本质是概率性的。

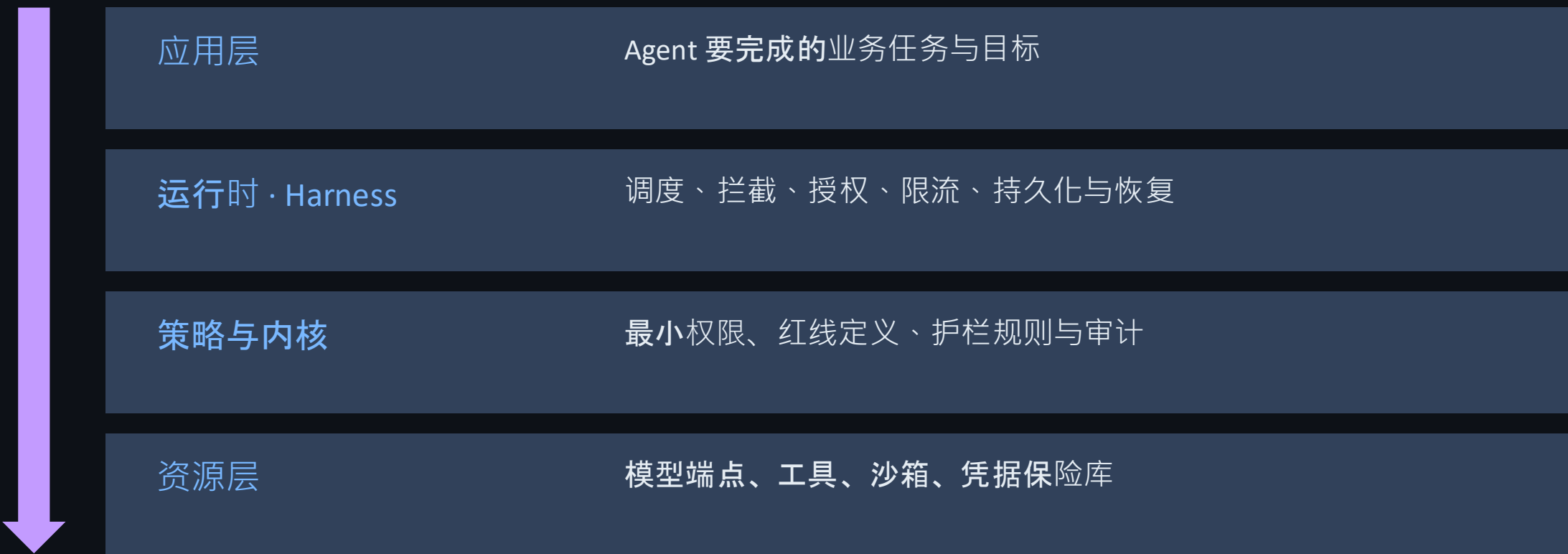
Harness 的职责 · 骨架

把不确定的意图变成确定的执行：类型校验、权限绑定、配额限流、状态落库、错误恢复、审批挂起。它保证的是「可控与可复现」。

一句话记忆：Model 负责「聪明」，Harness 负责「可靠」；混在一起，两个都做不好。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

像操作系统一样，Harness 是 Agent 的运行时底座



自上而下逐层调用：上层永远不能绕过下层的策略直接触碰资源，这正是操作系统的隔离思想。

来源：仓库 AgentHarness_HostedAgent（Brain/Hands/Session/Vault）

裸 LLM 直连业务系统，会撞上四类不可接受的风险

① 非确定性

同一输入可能给出不同、不可复现的动作

② 无状态

跨轮次遗忘上下文，重启即失忆

裸
LLM

③ 越权操作

分不清该不该做，容易触碰高危写入

④ 死循环

工具调用无人叫停，持续烧 token 与调用

这四类风险任意一条，都足以让企业拒绝把 Agent 接入生产系统。

非确定性让同一输入产出不可复现的动作



来源：Skill_Testing_Agent 对抗测试（确定性输出契约）

无状态与越权，让 Agent 既健忘又危险

无状态 · 健忘

裸模型只认当前上下文窗口。长任务里，早期的约束、**已完成的步骤**、审批结论都会随窗口滑出而遗忘，导致重复执行或推翻前序决定。

越权 · 危险

模型无法区分「能做」与「该做」。缺少运行时权限绑定时，一个测试意图可能拿着生产凭据，触发真实的高危写入。

健忘要靠状态持久化补，危险要靠权限绑定与护栏拦——都不是 prompt 能解决的。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

四类风险叠加，用一组示意数字看清代价

4

类风险同时存在

∞

失控循环的调用上限

0

裸模型自带的回滚能力

1

次越权写入即可致灾

这些数字是示意，用来传达量级直觉：风险不是线性叠加，而是彼此放大。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

剧本一：一次幻觉参数让批量删除脱离预期

触发条件

模型在缺少约束的工具里，把「清理过期数据」推理成无 where 条件的批量 delete。

直接后果

一次调用抹掉远超预期的记录；
无状态使它意识不到自己删多了，继续下一步。

Harness 对策

强类型 Schema 校验拒绝危险参数，
高危写入触发 HITL 审批挂起，
Trajectory 快照可回放定责。

同一个动作，有没有 Harness 拦截，是「被 review 挡下」还是「上生产事故」的区别。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

剧本二：越权调用把测试凭据用到了生产库

触发条件

任务本应连测试库，但凭据按逻辑名解析，运行时未绑定环境。

直接后果

读写打到生产库，越权操作产生真实且不可逆的副作用。

Harness 对策

运行时权限绑定 + Vault 按逻辑名注册，沙箱永远拿不到原始 token。

凭据不落地、权限运行时绑定，是把「越权」从可能变成不可能的关键设计。

来源：AgentHarness_HostedAgent · Vault 凭据管理

剧本三：工具循环无人叫停，烧光 token 预算

触发条件

工具持续返回可重试错误，模型不断「再试一次」，形成 Thought-Action 死循环。

直接后果

调用次数与 token 无上限增长，成本飙升，任务却毫无进展。

Harness 对策

调用频次配额 + 轨迹步数上限触发熔断，失败纳入自愈循环而非盲目重试。

没有配额与熔断，Agent 的「坚持不懈」就是账单上的无底洞。

来源：CodingAgent · 自愈编码循环 / 配额隔离

Agent Harness 的第一重与第二重：运行沙箱与资源调度

① 运行沙箱

把工具执行关进隔离环境，限定可访问路径与资源，副作用不外溢。



② 资源调度

统一管理模型端点、工具池与并发，按配额分配算力与调用额度。

先把「在哪跑、用多少」管起来：沙箱管边界，调度管资源。

来源：OpenClaw_AgentHarness · 沙箱与网关

第三重与第四重：安全策略执行器与持久化网关

③ 安全策略执行器

在每次工具调用前后强制执行策略：鉴权、参数校验、护栏过滤、红线拦截与结构化审计日志。

④ 持久化网关

所有状态经它落到 append-only 日志：Trajectory、审批结论、凭据引用，支撑重放、续跑与定责。

四重定位合起来，Harness 才既是「运行时」，又是「治理中枢」。

来源：AgentHarness_HostedAgent · Session 日志 / 策略执行

五大治理支柱，构成 Harness 的完整中枢



五根支柱同源治理中枢：接下来两大段逐根展开，每根都配真实机制与仓库出处。

支柱一至二

上下文与工具： 管住 Agent 的记忆与双手

Context 治理 · 工具拦截与沙箱隔离



上下文治理要同时对抗溢出与污染两种失效

Context Overflow · 溢出

长任务里对话与工具结果不断堆积，超出模型窗口，早期关键信息被挤出，任务失忆、跑偏。

Context Contamination · 污染

无关内容、错误观测或被注入的指令混入上下文，引发幻觉漂移，让后续推理越走越歪。

溢出是「装不下」，污染是「装错了」；两者都要靠主动的上下文治理来管。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

动态 Token 预算：给每一轮对话设定可算的上限

测算

统计系统提示、历史、工具结果各自占用的 token 份额。



分配

为每一部分设定预算配额，优先保证约束与目标不被挤出。



裁剪

超预算时按优先级裁剪低价值内容，把窗口留给当前目标。

把上下文当作一份有限预算来管理，而不是任其自然堆积。

滑动窗口裁剪：只保留与当前目标相关的最近轨迹

保留

最近若干轮

Thought/Action/Observation 与硬
约束原文完整保留。

压缩

较早的轮次摘要成要点，保留结
论、丢弃冗长过程。

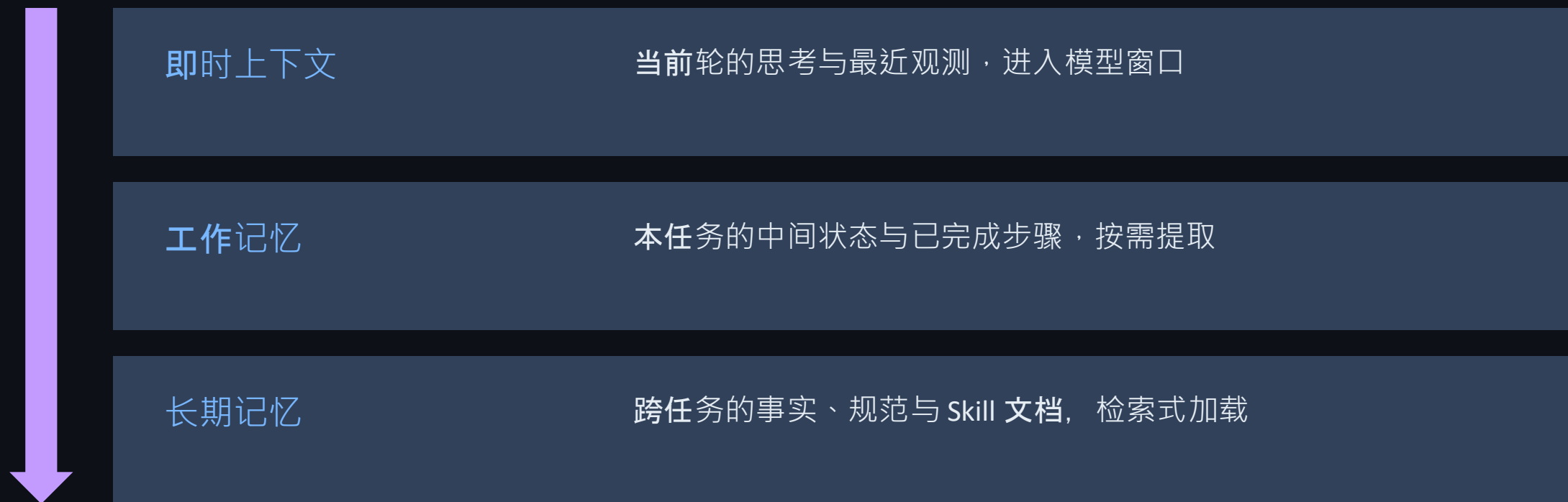
丢弃

已失效的中间产物与重复观测直
接移出窗口。

窗口随任务向前滑动：近处高清、远处摘要、无关即弃。

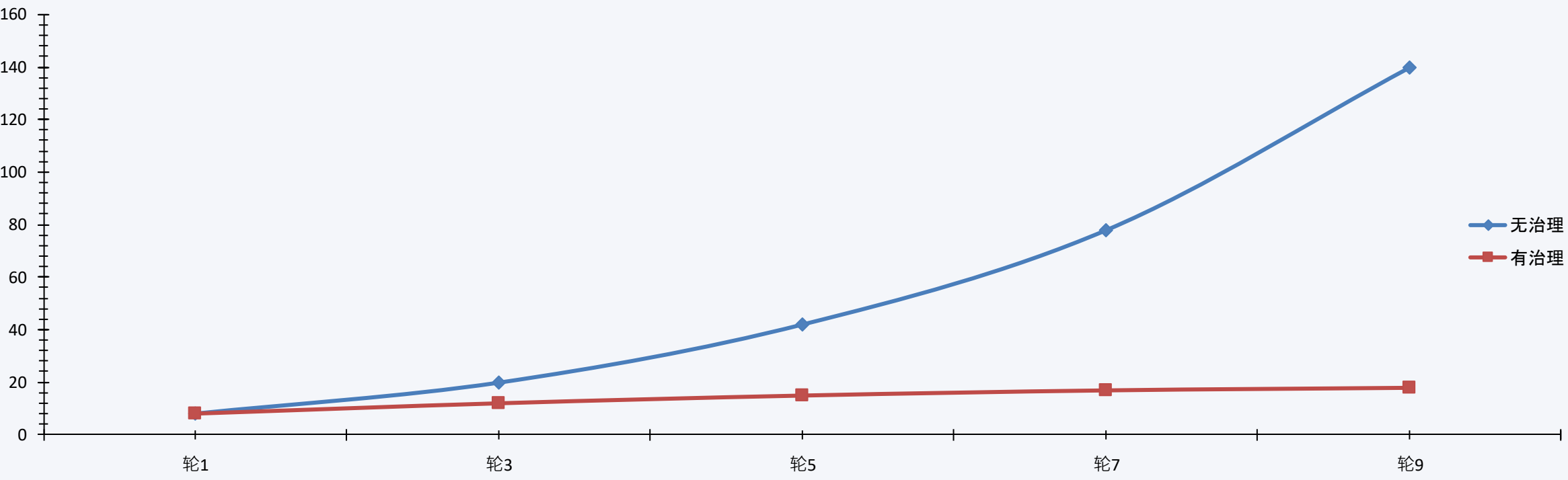
来源：模块三 Lab2 · 上下文滑动压缩过滤器

分层工作记忆：把长期事实与临时思考分开存取



不是所有信息都要进窗口：需要时再从对应层提取，用完即退。

不加治理，Token 会随轮次快速膨胀



相对 token 占用：无治理 8→140，有治理 8→18（示意数据）

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

CodingAgent 用 @FILE.md 按需加载 Skill 文档控制上下文

按需加载 · 上下文优化

coder Agent 只加载 spec 里用 @FILE.md 语法显式引用的 Skill 文档，让 prompt 窗口保持精简、可预测。

角色分工 · 自愈循环

coder / runner / diagnoser 各带严格工具白名单；
diagnoser 把 Python traceback 翻成结构化 Patch Plan 供下一轮消费。

上下文优化不是玄学：显式引用什么，才加载什么——其余一律不进窗口。

来源：仓库 CodingAgent/ 模块

四份 Skill 文档， 只在被引用时进入 prompt 窗口

Skill 文档	作用	何时进入上下文
PYTHON_STYLE.md	编码风格与命名规范	spec 用 @FILE 引用时
ALGO_PATTERNS.md	算法与数据结构范式	涉及算法实现时
ERROR_HANDLING.md	异常与边界处理约定	需要健壮性保障时
TESTING.md	测试用例与断言规范	编写测试代码时

同样的任务，只有相关文档进窗口——这是把 token 花在刀刃上的工程纪律。

来源：CodingAgent/workspace/skills/

工具拦截先做强类型 Schema 校验与入参清洗

强类型 Schema 校验

入参先过 schema：类型、范围、枚举、必填逐项检查，非法请求在进入工具前即被拒绝。

入参清洗

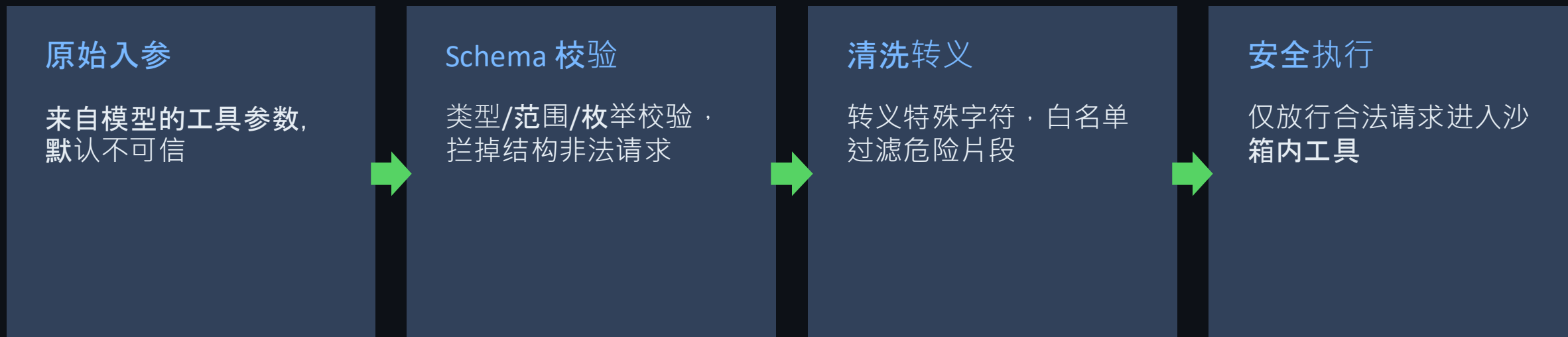
对字符串做转义与白名单过滤，阻断注入 payload 的构造与拼接。

失败即拒绝

校验不通过直接返回结构化错误给 Agent，纳入自愈循环而非放行执行。

工具边界是第一道闸：先证明请求合法，再谈执行。

SQL 与 Shell 注入必须在进入工具前被拦下



把「不可信输入」当默认前提，注入才没有可乘之机。

来源：模块二 课程大纲 · 注入防范

运行时权限绑定：用 OAuth / Entra ID 委托最小身份

身份委托 · OAuth / Entra ID

工具调用在运行时绑定最小权限身份，凭据由委托流下发，绝不写进 prompt 或代码。

最小权限 + 环境隔离

每只手只拿到完成任务所需的最小 scope；测试与生产凭据物理隔离，互不可见。

权限不是模型「自觉」出来的，而是运行时由 Harness 绑定与下发的。

来源：AgentHarness_HostedAgent · Vault / 身份委托

调用频次配额，给每只手设定可控的爆炸半径

N

每分钟调用上限

K

并发沙箱上限

T

轨迹步数上限

0

超限后的容忍度

N、K、T 由你的负载与预算设定（示意）；关键是超限即熔断，把爆炸半径钉死。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

OpenClaw 三 Agent 白名单：只有 Runner 能 exec

Agent	角色	工具白名单
A — Coder	读 SPEC.md, 写 solution.py	read · write · edit
B — Tester	读 spec + solution, 写 test	read · write · edit
C — Runner	沙箱内跑 pytest, 写 RUN_REPORT	read · write · exec

只有 Runner 拥有 exec， 且被 tools.exec.allowedPaths 限定在单一子目录——爆炸半径最小化。

来源：OpenClaw_AgentHarness/ 三 Agent 协作

支柱三至五

状态、安全与人机协同： 让 Agent 可恢复、可信、可控

断点续跑 · OWASP 护栏 · HITL 检查点



状态持久化让 Agent 从崩溃点续跑而非从头再来

Trajectory · 执行轨迹

把每一轮 Thought / Action / Observation 实时落库，形成可追溯、可重放的执行轨迹。

中间状态快照

在关键步之后保存状态快照，与模型上下文窗口解耦，进程崩溃也不丢已完成的进度。

状态一旦落库，重启就不再意味着重来——这是长任务能进生产的前提。

Trajectory 快照把每一轮思考与动作落库



轨迹 append-only：只追加不修改，任何一轮都可精确回放与定责。

时间轴上的每个点都是一次快照——崩溃后从最近的点续跑即可。

来源：AgentHarness_HostedAgent · Session（append-only 日志）

幂等重试：失败重放不会造成重复副作用

幂等键

每个副作用操作携带幂等键，重复提交只生效一次。

失败重放

从最近快照重放，而不是从任务起点整体重来。

无重复副作用

已提交的写入不再重复，杜绝重复扣款或重复删除。

可重试的前提是幂等：没有幂等键的重试，本身就是新的事故源。

断点恢复 Resilience Pattern：捕获错误并驱动自愈



错误不是终点而是输入：Harness 捕获它、喂回 Agent，促成自我修正。

来源：模块三 Lab3 · 错误自愈 (Self-Healing)

一次崩溃的完整恢复链路，端到端可追可续

阶段	Agent 状态	Harness 动作	产物
正常执行	运行中	逐轮写 Trajectory 快照	append-only 日志
崩溃 / 异常	中断	捕获错误，冻结最近快照	错误上下文
重启恢复	从断点续跑	wake(session_id) 重放日志	恢复到断点状态
幂等重试	再次执行	校验幂等键，跳过已完成副作用	无重复写入

每一阶段都有明确的状态、动作与产物——恢复因此是工程流程，而非碰运气。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

支柱四对齐 OWASP Agentic Top 10 的护栏体系

以清单为纲

对照 OWASP Agentic Top 10 逐项建立运行时护栏，而不是出事后再补。

输入侧防御

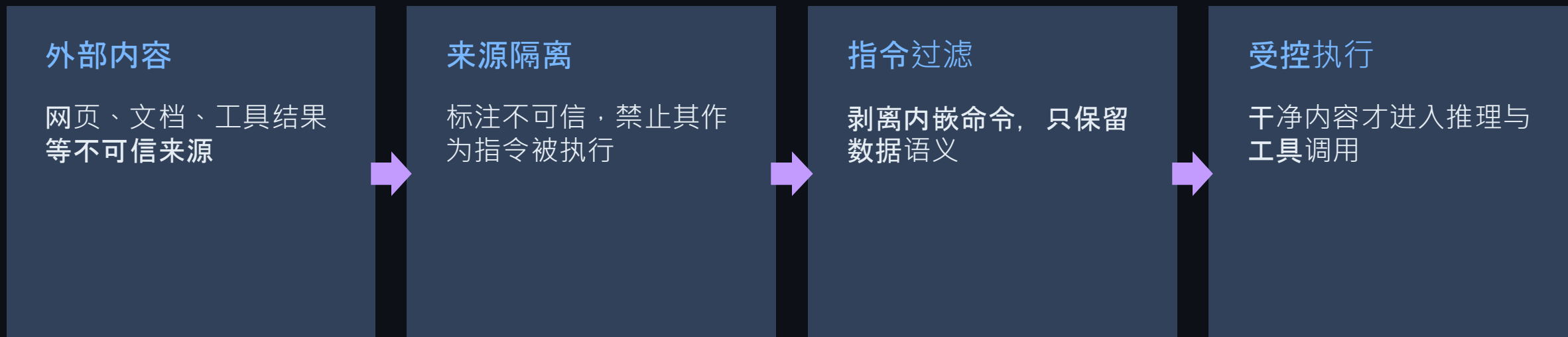
Prompt Injection 与间接数据投毒——清洗、来源隔离、不信任外部内容。

输出侧防御

内容安全过滤与 PII 脱敏——守住 Agent 说出口、写下去的边界。

安全不是某一处的开关，而是覆盖输入到输出的一整套运行时护栏。

Prompt Injection 与间接数据投毒是首要攻击面



把外部内容当数据、绝不当指令——这一条守住，注入就失去了入口。

内容安全过滤与 PII 脱敏守住输出边界

内容安全过滤 · Content Moderation

对模型输出与工具结果做安全分类，拦截有害内容后再交付下游或用户。

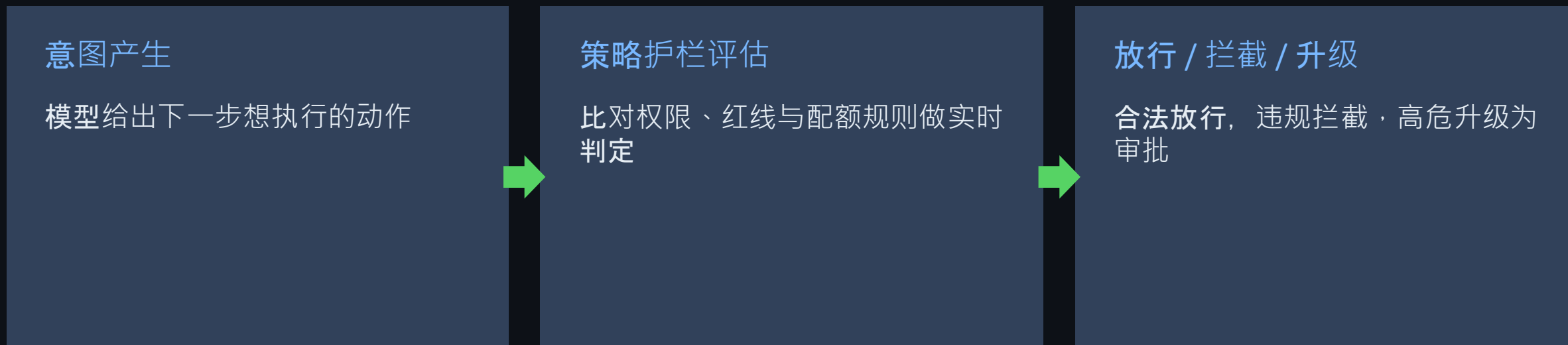
敏感数据脱敏 · PII Masking

输出前识别并脱敏身份证、密钥、账号等敏感信息，审计日志同样脱敏。

输入侧防被骗，输出侧防说漏——**两侧**都设护栏，边界才闭合。

来源：模块二 课程大纲 · 内容过滤与 PII 脱敏

非受控行为要用策略护栏而非事后补救



护栏在动作发生「之前」介入，才谈得上防御；事后审计只能定责。

来源：模块二 课程大纲 · 非受控行为防御

风险—对策矩阵：把每类威胁映射到具体防线

OWASP 风险	典型攻击面	Harness 对策
Prompt Injection	恶意指令劫持工具调用	输入清洗 + 工具白名单 + 护栏
间接数据投毒	被读取内容内嵌指令	来源隔离 + 外部内容不可执行
非受控行为	越权与高危动作	权限绑定 + 红线 HITL 拦截
敏感数据泄露	输出含 PII / 密钥	内容过滤 + PII 脱敏 + 凭据不落地

每一类风险都要能指着一条具体对策——指不出来的地方，就是缺口。

来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意

高危操作必须交回人类，这是企业的红线

高危写入

批量删除、schema 变更、数据覆盖等不可逆写操作。

资金流转

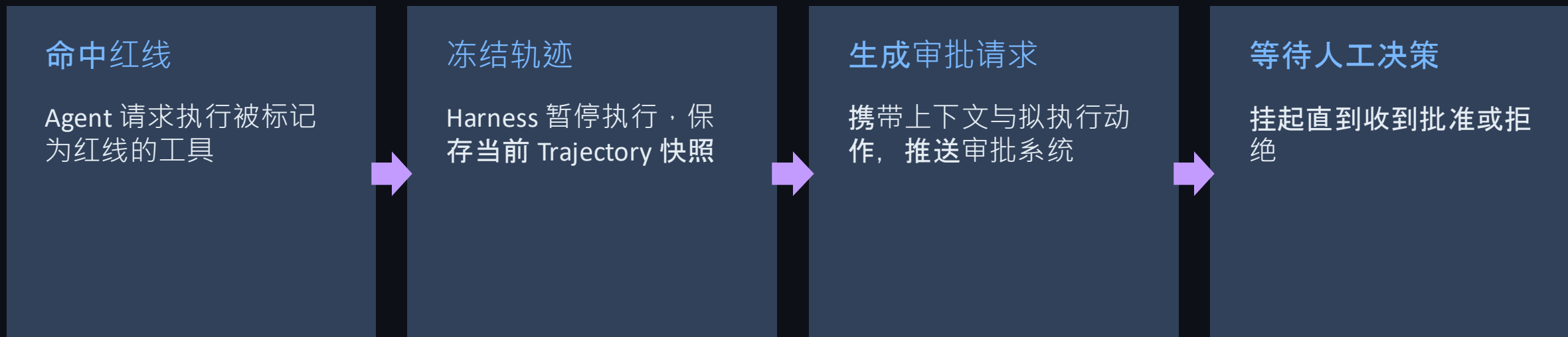
支付、退款、转账、额度调整等一切涉钱动作。

生产配置变更

部署、开关、权限、路由等生产环境配置修改。

先定义清楚「什么是红线」，HITL 才有明确的触发条件可挂钩。

HITL 把红线操作定义为可挂起的检查点



检查点让 Agent 在最危险的一步「停下来等人」，而不是先斩后奏。

来源：模块三 Lab3 · HITL 审批阻断

Suspend 挂起：Harness 冻结轨迹等待人工审批



暂停执行

命中红线即停，后续动作一律不放行



落库快照

把当前 Trajectory 与上下文写入 append-only 日志



释放资源

挂起期间不占用沙箱与算力，可长时间等待



等待回调

持有 session_id，等待审批系统回传决策

挂起不是阻塞进程，而是把状态安全落库、把资源还回去，随时可被唤醒。

Approval Webhook 回调与 Resume 恢复闭环

Approval Webhook · 审批回调

审批系统通过 webhook 回传决策 token；Harness 校验 token 与 session 后解除挂起。

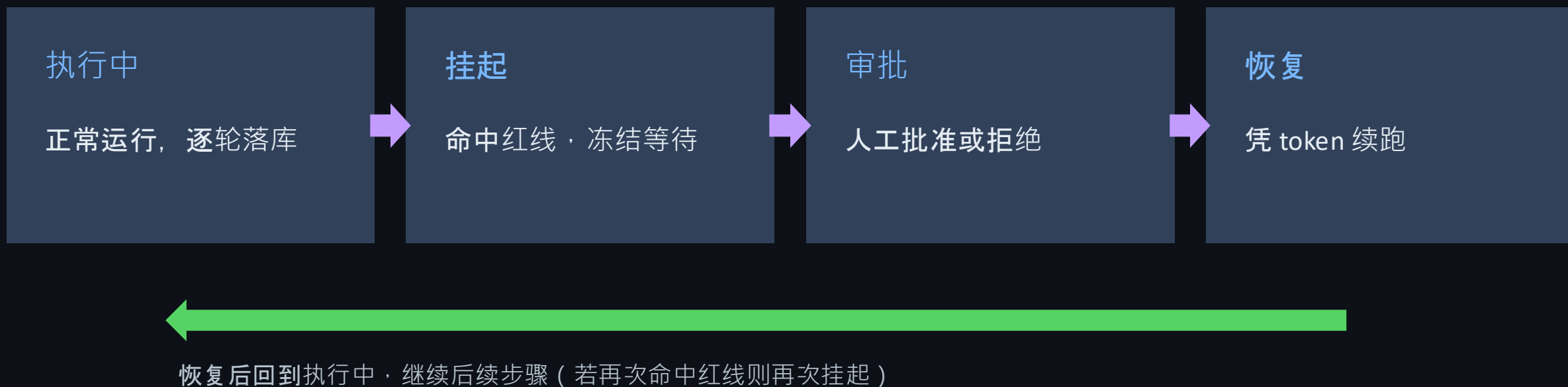
Resume · 状态恢复

从挂起点恢复执行，续跑后续步骤，整个批准—恢复过程写入审计日志。

从挂起到恢复形成闭环：人给决策，Harness 凭 token 精确唤醒对应会话。

来源：模块三 Lab3 · 注入审批 Token 驱动恢复

HITL 状态机：从执行到挂起再到恢复



整个 HITL 是一个循环状态机：正常跑、遇红线停、等审批、批准后续——安全与效率兼得。

来源：AgentHarness_HostedAgent · Suspend/Approval/Resume

五大支柱落到仓库的 Brain / Hands / Session / Vault 四层

Brain · 大脑

跑在 ResponsesHostServer 上的编排 Agent，负责决策与调度

Hands · 手

SandboxPool 里一次性、无状态的 worker，负责执行

Session · 会话

文件级 append-only 日志，支撑状态、重放与续跑

Vault · 保险库

凭据按逻辑名注册，沙箱永远看不到原始 token

五根支柱不是抽象口号：它们分别由这四层的具体机制承载。

Brain 只暴露五个工具，把编排权牢牢收在中心

Brain · SkillTestingHarness（编排）



execute

按名字派活给某只手，拿回结果



list_tools

列出可用的手（工具）



list_models

列出可用的被测模型



get_events

读取会话事件流



emit_note

向会话写入一条笔记

业务复杂度全在「手」里，Brain 只做编排——窄接口让大脑可控、可测、可替换。

来源：AgentHarness_HostedAgent · Brain 工具面

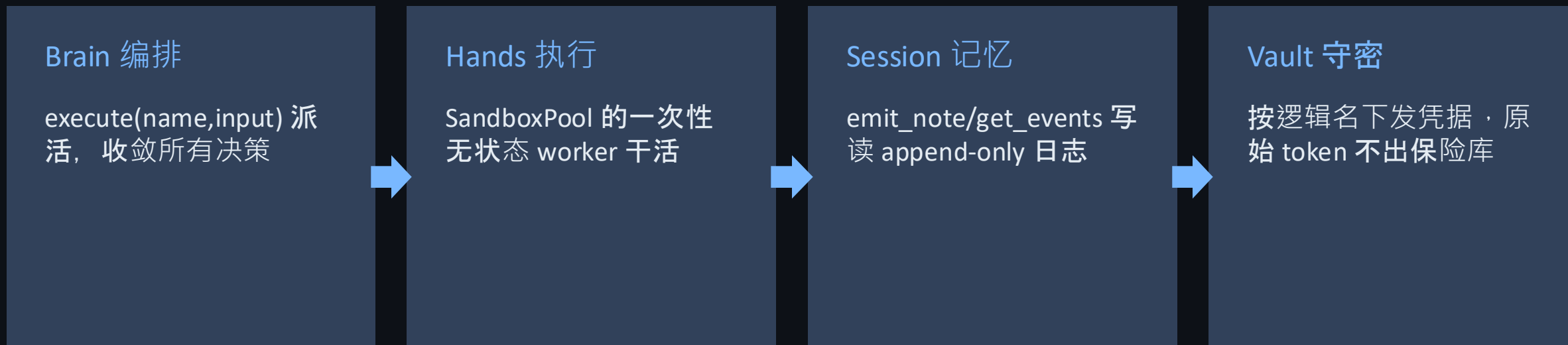
四层各司其职：一张表看清职责与关键机制

层	职责	关键机制
Brain 大脑	编排与决策	只暴露 execute / list_tools / list_models / get_events / emit_note
Hands 手	一次性执行	SandboxPool 中 7 只无状态的手，用完即弃
Session 会话	状态与重放	文件级 append-only 日志，可 wake(session_id) 重放
Vault 保险库	凭据安全	按逻辑名注册，沙箱永远看不到原始 token

Brain 决策、Hands 执行、Session 记忆、Vault 守密——职责单一，边界清晰。

来源：AgentHarness_HostedAgent · Brain/Hands/Session/Vault

Session 可重放、Vault 不落地：完整四层架构



Brain 向下写 Session、向 Hands 派活, Hands 经 Vault 取受控凭据——闭环且凭据不落地。

来源：AgentHarness_HostedAgent · 调用关系

Harness 设计原则：最小权限、可重放、可中断、凭据不落地



最小权限

每只手只拿完成任务所需的最小 scope 与工具白名单



可重放

状态写 append-only 日志，任何一步都能精确回放



可中断

红线动作能挂起等待人工，随时被唤醒续跑



凭据不落地

按逻辑名注册，沙箱与 prompt 永远看不到原始 token

这四条是把任意 Agent 送进生产的最低门槛——缺一条，就别急着上线。

模块二·小结

Model 让 Agent 会思考， Harness 让 Agent 能托付。

把五大支柱落进你的 Harness，下一站——模块三：为智能体挂载自定义 Harness 中间件，动手实战。



来源：EnterpriseAgenticWorkshop 仓库与课程大纲；数值为示意